```tsx
import React, {
  forwardRef,
  useState,
  useEffect,
  useRef,
  useCallback,
} from "react";
import { LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export interface NumericEditInnerIconConfig {
  icon: LucideIcon;
  className?: string;
}

export interface NumericEditActionButtonConfig {
  icon: LucideIcon;
  onClick: (value: number) => void;
  tooltipText?: string;
  disabled?: boolean;
}

export interface NumericEditProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange" | "value" | "defaultValue"
> {
  label?: string;
  error?: string;
  prefixSymbol?: string;
  f2Editable?: boolean;
  innerIcons?: NumericEditInnerIconConfig[];
  actionButtons?: NumericEditActionButtonConfig[];
  prefixIcon?: LucideIcon;
  suffixIcon?: LucideIcon;
  value?: number | string;
  defaultValue?: number | string;
  onChange?: (value: number) => void;
  decimalPlaces?: number;
}

export const NumericEdit = forwardRef<HTMLInputElement, NumericEditProps>(
  (
    {
      label,
      error,
```

```
    prefixSymbol = "",
    f2Editable = false,
    innerIcons = [],
    actionButtons = [],
    prefixIcon,
    suffixIcon,
    className,
    disabled,
    required,
    placeholder,
    value,
    defaultValue,
    onChange,
    decimalPlaces = 2,
    ...props
  },
  ref,
) => {
  const inputRef = useRef<HTMLInputElement>(null);
  const hasError = Boolean(error);
  const [isEditing, setIsEditing] = useState(!f2Editable);
  const [displayValue, setDisplayValue] = useState<string>("");

  const sanitizeValue = useCallback(
    (val: string): string => {
      if (!val) return "";
      const clean = val.replace(/[^0-9.-]/g, "");
      const parts = clean.split(".");
      if (parts.length > 2) {
        return parts[0] + "." + parts.slice(1).join("");
      }
      if (decimalPlaces > 0 && parts[1] && parts[1].length > decimalPlaces) {
        return parts[0] + "." + parts[1].slice(0, decimalPlaces);
      }
      return clean;
    },
    [decimalPlaces],
  );

  const parseToNumber = useCallback((val: string): number => {
    if (!val) return 0;
    const num = parseFloat(val);
    return isNaN(num) ? 0 : num;
  }, []);

  const formatDisplay = useCallback(
    (val: number | string): string => {
      if (val === "" || val === null || val === undefined) return "";
      const num = typeof val === "string" ? parseToNumber(val) : val;
      if (decimalPlaces > 0) {
```

```
      return num.toFixed(decimalPlaces);
    }
    return Math.round(num).toString();
  },
  [decimalPlaces, parseToNumber],
);

useEffect(() => {
  const initialValue =
    value !== undefined
      ? value
      : defaultValue !== undefined
        ? defaultValue
        : "";
  const formatted = formatDisplay(initialValue);
  setDisplayValue(formatted);
}, [value, defaultValue, formatDisplay]);

useEffect(() => {
  if (prefixSymbol) {
    setDisplayValue((prev) => prev);
  }
}, [prefixSymbol]);

const handleRef = useCallback(
  (node: HTMLInputElement | null) => {
    inputRef.current = node;
    if (ref) {
      if (typeof ref === "function") {
        ref(node);
      } else {
        ref.current = node;
      }
    }
  },
  [ref],
);

const handleKeyDown = useCallback(
  (e: React.KeyboardEvent<HTMLInputElement>) => {
    if (f2Editable && e.key === "F2") {
      e.preventDefault();
      setIsEditing(true);
      setTimeout(() => inputRef.current?.focus(), 0);
    }
    if (e.key === "Escape" && isEditing) {
      e.preventDefault();
      setIsEditing(false);
      inputRef.current?.blur();
    }
```

```
    if ((e.ctrlKey || e.metaKey) && e.key === "c") {
      const numValue = parseToNumber(displayValue);
      navigator.clipboard?.writeText(numValue.toString());
    }
  },
  [f2Editable, isEditing, displayValue, parseToNumber],
);

const handleChange = useCallback(
  (e: React.ChangeEvent<HTMLInputElement>) => {
    if (!isEditing) return;
    const sanitized = sanitizeValue(e.target.value);
    setDisplayValue(sanitized);
    const numValue = parseToNumber(sanitized);
    onChange?.(numValue);
  },
  [isEditing, sanitizeValue, parseToNumber, onChange],
);

const handleBlur = useCallback(() => {
  if (f2Editable && isEditing) {
    setIsEditing(false);
  }
}, [f2Editable, isEditing]);

const limitedInnerIcons = innerIcons.slice(0, 4);
const limitedActionButtons = actionButtons.slice(0, 4);

const numericValue = parseToNumber(displayValue);

return (
  <div className={cn("w-full", className)}>
    {label && (
      <label
        className={cn(
          "block text-sm font-medium text-text-main mb-1.5",
          disabled && "opacity-50",
        )}
      >
        {label}
        {required && (
          <span className="text-destructive ml-0.5" aria-hidden="true">
            *
          </span>
        )}
      </label>
    )}
    <div className="flex items-center w-full gap-2">
      <div
        className={cn(
```

```
              "relative flex items-center flex-1 min-w-0",
              "bg-card/90 backdrop-blur-[var(--backdrop-blur)]",
              "border-[color-mix(in_oklch,var(--color-border)_60%,transparent)]",
              "rounded-surface",
              "text-text-main placeholder:text-muted-foreground/60",
              "focus-within:ring-2 focus-within:ring-amber-500/20
focus-within:border-amber-500 focus-within:bg-amber-500/5",
              "disabled:opacity-50 disabled:pointer-events-none
disabled:cursor-not-allowed",
              "transition-all duration-200 ease-out",
              hasError &&
                "border-destructive/50 focus-within:ring-destructive/50
focus-within:border-destructive/50",
              disabled && "bg-muted/50",
              f2Editable && !isEditing && "bg-muted/30",
              f2Editable && isEditing && "bg-amber-500/5 border-amber-500",
            )}
          >
            {(prefixSymbol || prefixIcon) && (
              <div
                className={cn(
                  "flex items-center justify-center px-3",
                  "text-muted-foreground/60",
                  disabled && "opacity-50",
                  f2Editable && isEditing && "text-amber-500",
                )}
                aria-hidden="true"
              >
                {prefixSymbol && (
                  <span className="text-sm font-medium">{prefixSymbol}</span>
                )}
                {prefixIcon && (
                  <span className="w-4 h-4" aria-hidden="true">
                    {React.createElement(prefixIcon, { className: "w-4 h-4" })}
                  </span>
                )}
              </div>
            )}
            <input
              ref={handleRef}
              type="text"
              inputMode="decimal"
              className={cn(
                "flex-1 bg-transparent border-none outline-none",
                "px-4 py-2.5",
                "text-text-main placeholder:text-muted-foreground/60",
                "disabled:opacity-50 disabled:pointer-events-none
disabled:cursor-not-allowed",
                "w-full min-w-0",
                "text-right tabular-nums",
```

```
        "font-mono",
        prefixSymbol || prefixIcon ? "pl-0" : "pl-4",
        suffixIcon || limitedInnerIcons.length > 0 ? "pr-0" : "pr-4",
        f2Editable && !isEditing && "cursor-not-allowed",
        f2Editable && isEditing && "bg-amber-500/5",
      )}
      disabled={disabled || (f2Editable && !isEditing)}
      required={required}
      placeholder={placeholder}
      value={displayValue}
      onChange={handleChange}
      onKeyDown={handleKeyDown}
      onBlur={handleBlur}
      readOnly={f2Editable && !isEditing}
      {...props}
    />
    {suffixIcon && (
      <div
        className={cn(
          "flex items-center justify-center px-3",
          "text-muted-foreground/60",
          disabled && "opacity-50",
        )}
        aria-hidden="true"
      >
        <span className="w-4 h-4" aria-hidden="true">
          {React.createElement(suffixIcon, { className: "w-4 h-4" })}
        </span>
      </div>
    )}
    {limitedInnerIcons.length > 0 && (
      <div className="absolute right-3 flex items-center gap-1">
        {limitedInnerIcons.map((iconConfig, index) => (
          <span
            key={index}
            className={cn(
              "flex items-center justify-center",
              "w-5 h-5",
              "text-muted-foreground/60",
              iconConfig.className,
            )}
            aria-hidden="true"
          >
            <iconConfig.icon className="w-4 h-4" aria-hidden="true" />
          </span>
        ))}
      </div>
    )}
  </div>
  {limitedActionButtons.length > 0 && (
```

```
            <div className="flex items-center gap-1.5 shrink-0">
              {limitedActionButtons.map((action, index) => (
                <button
                  key={index}
                  type="button"
                  className={cn(
                    "relative flex items-center justify-center",
                    "w-9 h-9 rounded-md",
                    "text-muted-foreground/60 hover:text-foreground",
                    "bg-transparent hover:bg-accent",
                    "transition-transform duration-100",
                    "active:scale-95",
                    "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2",
                    "disabled:opacity-50 disabled:pointer-events-none",
                  )}
                  aria-label={action.tooltipText}
                  disabled={disabled || action.disabled}
                  onClick={() => action.onClick(numericValue)}
                >
                  <action.icon className="w-4 h-4" aria-hidden="true" />
                </button>
              ))}
            </div>
          )}
        </div>
        {error && (
          <p
            className={cn(
              "mt-1.5 text-sm",
              "text-destructive/90",
              "font-medium",
            )}
            role="alert"
          >
            {error}
          </p>
        )}
      </div>
    );
  },
);

NumericEdit.displayName = "NumericEdit";
```